Hosting on IIS & Proving It Is Safe

Which files you need, where they go, what each script does — and how to confidently show you did not disturb Teamcenter

A companion to the EMBRACE AI Hosting & Architecture Handbook

SIEMENS · Engineering Systems · AI CatalyESt
Edition · July 2026

1 The files you need

and exactly where each one goes

Hosting this app on IIS is **not** like hosting a plain static website, where IIS serves `.html` files directly. Our app is a live **Node.js program**, so IIS never runs the app — it only **forwards** requests to Node. That means two separate locations matter on the server.

ANALOGY · Real-life analogy

IIS here is like a hotel reception desk, and Node.js is the chef in the kitchen. Reception (IIS) does not cook — it just takes your order to the kitchen (Node) and brings the food back. Two different rooms, two different jobs.

A) The application files → C:\apps\embrace-ai

This is the whole Git repository — the actual app that Node runs. The important formats:

File / folder	Format	Why it is needed
`server.js`	.js (JavaScript)	The Node.js backend that runs the app
`package.json`	.json	Lists the Node libraries to install
`node_modules/`	folder	The installed libraries (from npm install)
`public/`	folder (.html/.css/.js)	The frontend the browser sees
`data/db.json`	.json	The database file
`deploy/`	folder (.bat/.ps1/.config)	Setup scripts + the IIS config

You put these on the server by cloning the repo there:

BATCH

```
cd C:\apps\embrace-ai
git clone https://code.siemens.com/engsys/ai_catalyest.git .
npm install
```

B) The IIS site folder → C:\inetpub\embrace-ai

IIS needs its **own** small folder containing **one** file: `web.config`. This is the only file IIS itself reads.

File	Format	What it does
`web.config`	.config (XML)	Tells IIS: enable WebSockets, and forward every request to http://localhost:3000

NOTE · In our project

Your repo already ships this file at `deploy/iis/web.config`. The `host-with-iis.bat` script copies it into `C:\inetpub\embrace-ai\web.config` for you — you do not place it by hand.

2 How the IIS process works

the journey of one request

Once set up, here is what happens each time someone opens the app:

1. **Browser makes a request**
   A colleague opens http://SN1W7220:8080 in their browser.
2. **IIS receives it**
   The request lands on the new "EmbraceAI" IIS site, listening on the dedicated port 8080.
3. **web.config rewrites it**
   The rewrite rule says: forward this request to http://localhost:3000.
4. **ARR forwards to Node**
   The Application Request Routing module actually passes the request to Node.
5. **Node runs the app**
   server.js (on 127.0.0.1:3000) processes it and reads/writes db.json.
6. **The answer returns**
   Node replies to IIS, and IIS sends it back to the browser.

In short: **Node files live in C:\apps\embrace-ai and run as a background service; IIS files live in C:\inetpub\embrace-ai (just web.config), and IIS is a forwarding front door.** The script wires the two together.

Prerequisites IIS needs (installed once)

- IIS with the **WebSocket Protocol** feature
- **URL Rewrite** module
- **Application Request Routing (ARR)** module

TIP · Good to know

The `host-with-iis.bat` script checks for URL Rewrite and ARR and clearly warns you if either is missing, so you are never left guessing.

3 What each script does

host-with-iis vs host-without-iis

host-with-iis.bat (behind IIS)

1. **Checks admin + prerequisites**
   Confirms Administrator rights and that the app and NSSM exist.
2. **Enables IIS WebSocket feature**
   So live quizzes work through the proxy (safe, repeatable).
3. **Checks ARR + URL Rewrite**
   Warns you only — does not force anything.
4. **Turns on ARR proxy**
   Enables server-level reverse proxying.
5. **Sets Node to 127.0.0.1:3000**
   The app listens privately; only IIS can reach it.
6. **Creates a NEW IIS site**
   A separate "EmbraceAI" site on dedicated port 8080.
7. **Opens firewall + verifies**
   Allows the port and tests that the app answers.

host-without-iis.bat (direct Node)

1. **Checks admin + prerequisites**
   Same safety checks.
2. **Confirms the port is free**
   Uses netstat to ensure port 8080 is not already busy.
3. **Sets Node to the public port**
   Node answers the browser directly on 8080.
4. **Ensures auto-restart**
   Service auto-starts on boot and restarts on failure.
5. **Starts service + firewall**
   Brings the app up and allows the port.

6

Verifies

Tests that the API answers. IIS is not touched at all.

4 Proving you did not break IIS

five facts you can stand behind

If a teammate says *"IIS on 7220 doesn't seem to work — did you do something?"*, here is what you can calmly and truthfully say. Each point is true **by design** of these scripts.

NOTE · Fact 1 — no IIS shutdown

1. "I never stopped, disabled, or reset IIS." The two new scripts contain no iisreset, no Stop-Service W3SVC, and no disabling of IIS services. They only ADD things.

NOTE · Fact 2 — existing sites untouched

2. "I never touched any existing site or the Default Web Site." The script only runs appcmd add site for a brand-new "EmbraceAI" site. It never edits, stops, or deletes Teamcenter's site or any binding they use.

NOTE · Fact 3 — no port conflict

3. "I used a separate, dedicated port." Teamcenter uses port 80; EmbraceAI uses 8080 (and Node privately on 3000). Different ports cannot conflict — and the direct script even checks the port is free first.

NOTE · Fact 4 — fully reversible

4. "Everything is reversible in seconds." Removing our footprint entirely has zero effect on anything else (commands below).

NOTE · Fact 5 — auditable

5. "It is all in Git — fully auditable." Anyone can read deploy/windows/host-with-iis.bat and confirm there is no destructive command.

The complete removal (reversal) commands

BATCH

```
appcmd delete site "EmbraceAI"
nssm stop EmbraceAI
netsh advfirewall firewall delete rule name="EmbraceAI HTTP 8080"
```

Deletes only our site, stops only our service, removes only our firewall rule. Teamcenter is untouched.

How to actually diagnose IIS health

Run these on the server — they prove IIS's own services and their sites are healthy, independent of ours:

POWERSHELL

```
Get-Service W3SVC, WAS | Select Name, Status     # should be Running
Get-Website | Select Name, State, Bindings        # their sites Started
```

If W3SVC is Running and their sites are Started, IIS is fine — and any problem with *their* site is unrelated to a script that only added a separate site on a separate port.

5 The one honest caveat

full transparency with your team

To be completely straight, there is exactly **one** IIS-level change in `host-with-iis.bat` that is not scoped only to our site: **step 4 enables ARR's server-level proxy flag** (`system.webServer/proxy enabled=true`). This is a global IIS setting. It is normally harmless — it simply allows proxying — but it is server-wide, so mention it for full transparency:

WARNING · Say this, for honesty

"The only server-wide change is enabling ARR proxying, a standard additive setting. I made no changes to any existing site, port, or the IIS services themselves."

If your team would prefer you make **zero** server-wide changes, use `host-without-iis.bat` instead — it touches IIS **not at all** (Node simply runs on its own port). That is the easiest position to defend: *"I didn't go near IIS."*

TIP · Good to know

Bottom line: with the direct script you can say you never touched IIS. With the IIS script, the only global change is the standard ARR proxy toggle, and everything else is a brand-new, separate, reversible site on its own port.

You are covered: both scripts are additive, reversible, and auditable — your existing IIS workloads keep running throughout.